

Agentic engineering leads every engineer down one single path. Better agents, more agents, and then custom agents. If you can prompt engineer and context engineer a single agent successfully, then the next step is to add more agents. Scale your compute to scale your impact. Then once you learn to delegate work to sub agents and new primary agents, there's only one place left to go. Custom agents. But why isn't CLAUDE Code, Codec CLI or the Gemini CLI enough? The out of the box agents are incredible, but there's a massive problem with these tools. They're built for everyone's code base, not yours. This mismatch can cost you hundreds of hours and millions of tokens scaling as your code base grows. This lesson is about flipping that equation. Here we master custom agents so your compute works for your domain, your problems, your, your edge cases. This is where all the alpha is in engineering. It's in the hard, specific problems that most engineers and most agents can't solve out of the box. In this advanced lesson, we showcase eight unique custom agents built on the CLAUDE code SDK. Each one showing you how to deploy across your stack products and engineering lifecycle. The agents are on the horizon. It's time to master agents by going all the way to the bare metal to the custom agent so you can scale your compute far beyond the rest in the building. Specialized agents code base Inside of apps, you can see eight custom agents. Let's start simple with the pong agent. So here's a simple agent that's going to showcase the most important aspect of custom agents you can see. We have a simple, concise user prompt and agent response. And we have some session stats. Let's run this again. All right, let's go ahead and change our prompt. Hello, simple prompt. We're just getting started with custom agents. I said hello. It just responded with pong. Okay, summarize this code base. Okay, there's my user prompt. And once again, the PONG agent is responding. Again with pong. What's going on here? Let's ask it. What's happening? Can you do anything other than pong? Okay, there's a user prompt and. Okay, so all this agent does is pong. Type one more here. Just going to write ping. This silly agent encapsulates the most important concept when you're building custom agents. No matter what we prompt here, the response is always pong. Why is that? Let's dive into this file. So the first most important concept is none other than the system prompt. We are using the cloud code SDK and we're setting up Only two things. We're modifying just two aspects of this agent, but it changes everything. So you can see cloud for Sonnet and here the system prompt. So let's open up the system prompt. You can see we have this dedicated load system prompt method. There's the path to the prompt. Let's open it. We have completely overwritten the claude code system prompt with this title of our agent purpose. And we have a simple three line instruction. You are a PONG agent. Always respond exactly with pong. That's it. The system prompt is the most important element of your custom agents. With zero

exceptions. We are modifying the core four, specifically the prompt. But now we have two very important prompts to pay attention to. System prompts and user prompts. The system prompt affects every single user prompt. The agent runs every single one. So all of your work is multiplied by your system prompt. So how does the cloud code SDK work? Let's go ahead and break down the key ideas. The SDK works like this. You set up your options, you set up your agent. Here we're just running the query and then you handle the response. The cloud code SDK is a powerful tool for putting together agents. As you'll see as we progress, all the pieces are there and they're incrementally adoptable. And then lastly, we're just doing some logging. That's it. That's the end of our agent. Right? A simple 150 line Pong agent. Most of this is just logging. We're setting up the agent. We're then querying the agent. We're then handling the response of the agent. And take note of these specific blocks that you can report. We have our agent message blocks, we have a result message and you can parse specific information out of these as you'll see as we progress. But this is the PONG agent and this showcases the power of the system prompt. Remember all that work that the Claude code team has put into making a great agent, right? The Claude code agent that you know and love is now gone. Okay, you have to be very careful with the system prompt. We now have a new product. This is not cloud code anymore. Okay. Might be using the same model, but the system prop is truly what builds the agent. Now, of course, we are still using the tools that's important to call it here, as you'll see as we progress into more and more capable custom agents. Let's move on to our second agent, the Echo agent. Inside of our Echo agent, you can see a similar structure. We have a single Python script here that we're going to execute `UV run echo agent py`, and we're going to see a similar structure. Right? We have our echo agent kicking off user prompt, agent response. But here we have something different. Here we have, of course, a custom tool. So our agent is saying, I'll use this tool. And then we have this tool call block here. And then we have the response of the tool call, and then our agent response. All right, so these are all unique elements coming out of our agent. You want to be keeping track of these, right? Keep track of the core four. If you understand the core four and how each element flows and controls your agent, you will understand compute and you'll understand how to scale your compute. Let's go ahead and echo ourselves. So I'm going to say echo this string and then I want it in reverse, in uppercase, repeat two times. Here we're just playing with the tool that our custom agent has. All right, so there's the user prompt, there's that call in reverse. This text comes out to. Custom agents are powerful. We ask it to repeat and go uppercase. So that's exactly what our agent has done. Okay, we're scaling up a little more. We're adding a few more capabilities to our agent as we explore custom agents. Let's understand what's going on here in the echo agent. Same structure. I always love collapsing everything, and you can just quickly understand what's going on. We have main. We have a tool and we have our system prompt. Always look for the system prompt and then look for the custom tools. We have a couple parameters here. Then we have something special. So once again, you

can always just search cloud code options. We have an MCP server built in this script. Create SDK, MCP server, and we're passing in a single tool. So this builds an entire MCP server in memory for our agent. This is super powerful. So let's look at our Echo tool and understand what it does. Tools for your cloud code SDK, they're built like this. A decorator. We then have the name, we have the description. So keep in mind the description of your tool tells your agent how to use it. In addition to the actual parameters that get passed into your tool and the actual arguments, you just pass in a dict, and then you do whatever you want to do inside of your tool. Here we step back into traditional deterministic code. Right? It's a tool call. Do whatever, whatever you want, but be sure to respond in the proper format. You can see here, here's our log. Here's the transforms we were doing. Right. Just a small example tool call for our agent. This is a custom agent with a custom tool set. Now, something interesting here as well. You can see we are running Claude Haiku. We have downgraded our model to a cheaper, less intelligent, but much faster model. Right. This is a simple agent. It doesn't need powerful intelligence. So we've dropped down the model. As you progress through each agent, keep in mind we have the 12 lever points of agent decoding that we're paying attention to. But whenever you see cloud code Options, isolate the Core 4. How will the Core 4 be managed given this setup? Now, we have something else here that's important to call out. We're using the CLAUDE SDK client class instead of our PONG agent, where we were just using that query command. All right, so query is for one off prompts and the CLAUDE SDK client is for continuous conversations. Let me show you exactly what I mean here. Let's clear this and let's run this once again with follow up. So if we kick this off, our agent is going to run that same prompt like normal, but then it's going to continue the conversation. We're going to pass in another follow up prompt. I'm going to say, concisely summarize our conversation in bullet points. And so here it is, right? Our agent quickly responds, follow up Response, concise summary. 1, 2, 3, 4. Our agent has kept track of the conversation. Because we're using the Claude SDK client and because we've written multiple query calls, you can see we have two here, follow up prompt and the original user prompt. All right, we have both the tool use block that we care about and we have the text block. You can stream all the messages coming back out from your agent after you query and just take them in. Right? Take them in, do whatever you want with them. Here, of course, we're just logging with clean rich panels, concisely communicating what's happening with our agent. If you don't know what your agent is doing, if you don't adopt your agent's perspective, it will be hard to improve them and tweak them and manage them. So that's what we're doing here. And you can see the follow up prompt. We fired off that additional query and we just looked for that one response. Right. We're looking for a tool use block and really we only were looking for the text block here. Right. This is not actually that useful. This is a custom agent with a few more capabilities. Right. We've specified the model. We've given it a custom tool to work with. But here's something interesting that if you're using these agents left and right, you probably haven't been paying attention to. If we run this

same agent and we just drop everything down here and we just say list your available tools. We are actually still running a lot of the Claude code baked in tool set. So check this out, right? These are all the cloud code tools plus our tool. And so everything that's going into your agent winds up in the context window at some point. We have 15 extra tools, 15 extra options that our agent has to choose from and select to do the job. We've asked it. Now our Echo agent does not need any of these tools. All right? And so as we progress, we're going to fine tune, we're going to get more control over our agent. If you ever run the slash context commander, understand what's going into your agent. You know that these 15 tools consumes context. It consumes space in your agent's mind. We're going to learn how to control that knob in our next agent. The last thing we want to point out here is our echo system prompt. Okay? Very simple, right? Again, it's important to call out, this is not the Claude code product anymore. As soon as you touch the system prompt and, and once you start dialing into the tools, you change the product, you change the agent. All right, so you can see here, title, purpose, echo agent, use the tool when asked. All right, so there's not a lot happening here, but there is a lot that was happening that isn't happening anymore. Right? If that makes sense. We have blown away the Claude code system prompt. We have completely overwritten it. Now it is important to mention you can append system prompt, right? You can just add on to what's already there. This is also powerful. This is more of a way to extend cloud code versus overwrite it and build a true custom agent. All right, let's continue, let's progress, let's get more control over agents. Let's scale this up and let's look at a new form factor for agents. Now we arrive at custom 7 micro SDLC agent. Let's understand the structure of this code base, back end, front end, reviews, specs and a couple of test files. So let's go ahead and just kick this off. We already have this running here. Back end, front end. Let's see exactly what this is. You probably understand what's going on here, right? We have an agent by agent task board where we hand off work from one Agent to another agent in a kind of micro software development life cycle. Now you can see here we have a couple tasks already shipped. We have some aired ones over here. Let's just go ahead, create a new task. We're going to do something really stupid and simple. I just want to showcase the workflow and showcase the capabilities that you can build into your user interfaces with agents. So update titles is just going to be super simple. Update the HTML title and header title to be plan, build, review and ship agents. And now we have that here, right? Update titles and we just drag it into plan. And now this workflow is going to kick off. Let's go ahead and refocus on the actual agent running on the back end. So let's just collapse the front end there and let's see what our agent is doing. The difference here is we have a multi agent system, right? We're not just deploying a single agent anymore, we're deploying multiple agents inside of our ui. So now we have a planner agent. It's noticing, it's thinking, right? We have planner, thinking planner tool calls, hook interceptions, right? Tool allowed. We have permission system inside of this. It has the full structure. And now it's going to start updating this code, right? It's a simple

title change. This isn't, you know, rocket science. But we should see our title update here and we should see our HTML title update as well, right? It is operating on itself. Couple of cool things, right? You can kind of build out your own multi agent orchestration system however you want. We have the messages on the left or the total messages getting streamed to the front end via websockets. And then we have the total tool calls on the right. Okay? So our agent is working. You can see there the planning is incrementing. But you can see here our builder agent is now processing. And this is really cool, right? Our build agent is now processing. So the planner is done. You know, you'll notice the UI is going to be a little kind of glitchy here because it is updating on itself. And so it's, you know, actively starting and restarting the server and the client. But that's fine. Our websocket should pick this up and continue processing. So there you can see the spec. You know, we're operating on that plan. We're operating in the review now. There we go. We can see that we're in the review step now. Okay. So very cool way to just kind of see how agents can work together. And then you just package it in a Nice ui. Then you can do more than you ever could before, frankly. Right. We have a dedicated workflow working step by step by step. Plan, build, review and handing it off to the next stage in the process. Okay, you can see here our reviewer agent. There's all of its tools and you can see it just operating, doing work for us. Right? We're using a lot of the out of the box cloud code tools, right? If you don't need to reinvent the wheel, don't do it. Let's be super clear about this. The cloud code SDK running the base Claude code agent is incredible. We've been using it since cloud code was released. You can see here, we just moved to shipped. There we go. Update titles. And you can see here, guess what happened? Plan, Build, Review and Ship Agents. Both titles updated. This work shipped with a single prompt outside the loop. Right? This is a perfect example of an out loop review system using the Peter framework. We dialed into this a ton inside of Tak. If you haven't finished the Tak lessons, you should probably stop watching this video immediately and finish tack. It's all about transitioning from in loop to outloop to eventually zte. The agent is compute scaled. You want to know how to deploy this across your tools, products, services for yourself, for your engineering work and for your users and customers. And the cloud code SDK and other agent SDKs. These are all powerful toolkits you can use to control the core for within your agents to scale them intelligently. If you're doing one size fits all work, use the out of the box agent. Don't think super hard about this. Just deploy compute to get the job done. But as your work becomes more specialized, as you deploy agents across all aspects of your engineering, you first want better agents, more agents and then custom agents. This is a code base you're going to want to ultra think about and understand how you can extend and use the ideas we've discussed here. Deploying effective compute and deploying effective agents is all about finding the constraints in your personal workflow and in your products. Think about repeat workflows that can benefit from an agent inside of a script, a data stream, an interactive terminal, all the way up to user interfaces that can tackle high return on investment problems in your tools, for

your team and for your business. Agentic coding is not so much about what we can do anymore, it's about what we can teach our agents to do. This is how we push our compute to its useful limits. Follow this path to unlock massive value for your tools and products. Better agents, more agents, and then custom agents. I'll see you in the next agentic horizon extended lesson.